Flask Tutorial

What is Flask:

Flask is a web framework that provides libraries to build lightweight web applications in python. Flask is considered as a micro framework. It is based on WSGI toolkit and jinja2 template engine.

What is WSGI:

WSGI(Web Server Gateway Interface) is an interface between web servers and web apps for python. mod_wsgi is an Apache HTTP server module that enables Apache to serve Flask applications.

What is Jinja2:

Jinja2 is a web template engine which combines a template with a certain data source to render the dynamic web pages.

Features of Flask:

- Fast Debugging
- Restful Request Handling
- Application Programming Interface
- Flexible Configuration
- Integrated Unit Testing

Advantages of Flask:

- Routing URL is easy
- Database integration is easy
- Easy to develop and maintain applications
- Higher compatibility with latest technologies
- Technical experimentation
- Easier to use for simple cases
- High scalability for simple applications
- Small core and easily extensible
- Lots of resources available online especially on GitHub

Disadvantages of Flask:

- Higher maintenance costs for more complex system
- Complicated maintenance for larger implementations
- Async may be a little problem
- Lack of database and ORM
- Offers limited support and smaller community compared to Django
- Setting up a large project requires some previous knowledge of the framework

Creating First Application:

```
from flask import Flask
app = Flask(__name__)
@app.route('/')
def hello_world():
   return 'Hello World'
if __name__ == '__main__':
   app.run()
```

Explanation:

- Flask(__name__) creates the Flask application
- @app.route('/') connects the home URL(/) to a function
- The function returns text shown in the browser
- App.run() starts the local server

Flask App Routing:

Flask App Routing is the process of mapping URLs to specific functions in a web application. When a user enters a URL in the browser, Flask checks the defined routes and executes the corresponding function to generate a response. This is done using the @app.route() decorator, which binds a URL path to a function.

Dymanic URLs:

We can also build dynamic URLs by using variables in the URL. To add variables to URLs, use the <variable_name> syntax in the route. The function then receives the <variable_name> as keyword argument.

Change Datatype:

We can also use a converter to convert the variable to a specific data type. By default, variables are treated as strings. To specify a data type, use <converter:variable_name>. The following converters are supported:

- string – Default type and it accepts any text without a slash
- int – accepts positive integers
- float – accepts positive floating-point values
- path – a string but also accepts slashes
- uuid – accepts UUID strings

add_url_rule() Function:

The URL mapping can also be done using the add_url_rule() function. This approach is mainly used when the view function is defined in another module. In fact, the app.route calls this function internally.

Syntax – add_url_rule(rule, endpoint, view_func)

Flask- HTTP Method:

HTTP methods define how a client(browser) interacts with a server in a web application. In Flask, they are used to handle different types of requests like fetching data, sending data or updating resources. Common HTTP Methods –

- GET – To request data from the server.
- POST – To submit data to be processed to the server.
- PUT – Replaces the entire resources with new data. If doesn't exist, a new one is created.
- PATCH – Updates only specific parts of a resource without replacing the whole thing
- DELETE – Deletes the data on the server at a specified location.

GET Method:

The GET method is used to request data from a server. It appends data to the URL in a name-value pair format. GET should not be used for sensitive data since URLs are visible in browser history.

request.args.get('num') - retrives the number from the URL paramters.

POST Method:

The POST method is used to send data to the server for processing. Unlike GET, it does not append data to the URL. Instead, it sends data in the request body, making it a better choice for sensitive or large data.

request.form.get('num') – retrives the number from the form.

Flask- Variable Rule:

Flask variable rules allow us to create dynamic URLs by defining variable parts within the route.

A variable rule is defined using <variable-name> within the route.

The captured variable is automatically passed as a keyword argument to the associated function.

Dynamic URLs Variable in Flask:

The following converters are available in Flask-Variable Rules:

- String – Default, It accepts any text without a slash
- Int – accepts only integers
- Float – like int but for floating point values
- Path – like the default but also accepts slashes
- Any – matches one of the items provided
- UUID – accepts UUID strings

Flask-Redirect to a URL:

A redirect is used in the flask class to send the user to a particular URL with the status code. Opposite way, this status code additionally identifies the issue. When we access a website, our browser sends a request to the server, and the server replies with what is known as the HTTP status code, which is a three-digit number.

Syntax – flask.redirect(location, code=302)

Where,

- location(str) – The location which URL directs to
- code(int) – The status code for Redirect
- Code – The default code is 302 which means that the move is only temporary

Code	Status
300	Multiple_choices
301	Moved_permanently
302	Found
303	See_other
304	Not_modified
305	Use_proxy
306	Reserved
307	Temporary_redirect

url_for() Function in Flask:

Another method you can use when performing redirects in Flask is the url_for() function URL building. This function accepts the name of the function as the first argument, and the function named user(name) accepts the value through the input URL. It checks whether the received argument matches the "admin" argument or not. If it matches, then the function hello_admin() is called else the hello_guest() is called.

Flask Errors:

Flask provides the abort() function to stop request processing and return an HTTP error response. It is commonly used a requested resource does not exist, the request is invalid, or access is not allowed.

Syntax – abort(code, message=None)

Where,

- code(int) – The HTTP status code that represents the type of error
- message(str, optional) – A custom error message that can be returned with the response

Code	Error
400	Bad request
401	Unauthenticated
403	Forbidden
404	Not Found
406	Not Acceptable
415	Unsupported Media Type
429	Too Many Requests

How to Change Port in Flask app:

The default port for the Flask application is 5000. We may want to change the port may be because the default port is already occupied. To do that we just need to provide the port while running the Flask application. We need to use below command to run the Flask application with a given port.

```
if __name__ == '__main__':
        app.run(debug=True, port=port_number)
```

Changing Host IP Address in Flask:

By default, Flask runs on 127.0.0.1:5000, which means it can only be accessed from the same machine. However, we may want to access our Flask app from other devices on the same network or even from the internet. To update the host IP address, we need to use host parameter.

```
if __name__ == '__main__':
        app.run(host='192.168.0.105')
```

Flask – Templates and Static Files:

Flask uses templates to create dynamic web pages with the Jinja2 templating engine. It also covers how static files such as CSS, JavaScript and images are organized and used to style and enhance Flask applications.

Flask Template:

A flask template is a template of a webpage that can be used by web developers to reduce the hectic work of writing the code for each webpage of a website.

render_template() method:

The render_template() method, allows us to fetch the HTML files, and pass them to the browser.

Template Inheritance in Flask:
Template inheritance is a powerful feature in Jinja, the templating engine used in Flask. It allows us to define a common structure for web pages, such as headers, footors and navigation bars, in a base template. This prevents redundant code and makes managing multiple pages easier.

Features of Template Inheritance:
- Code Reusability
- Better Maintainability
- Separation of Concerns
- Scalability

Flask-Jinja Template Inheritance:
Instead of reusing full templates, we can inherit them using Jinja blocks. Here's how –
- Create a block in a template using {% block <name> %} …. {% endblock %}
- This allows other templates to extend it and replace the block content.
- Everything above {% block body %} stays the same in all templates, while content inside the block can change.
- The block ends with {% endblock %}, allowing templates to override only specific sections.

Flask – Static files:
In Flask, static files refer to files such as CSS, JavaScript, images, videos and audio files that do not change dynamically. Flask provides a built-in way to serve these static files using the/static directory.

Flask-form Handling:
This section explains how Flask handles forms, including capturing user input using the request object and managing validation and security with Flask-WTF. It also covers file uploads and CSRF protection.
- Flask-WTF
- Handling File Uploads
- CSFR Protection

Flask-WTF:
Flask-WTF is a flask extension that integrates the WTForms library, making form creation and validation easier in Flask applications. It provides a structured way to build forms, handle validation, and render them in HTML.
In Flask-WTF, forms are defined as classes that extends the FlaskForm class. Fields are declared as class variables, making form creation simple and structured.
Common WTForms Field Types:
- StringField – Text input field for string data.
- PasswordField – Input field for password values.
- BooleanField – Checkbox for True/False selection.
- DecimalField – Input field for decimal values.
- RadioField – Group of radio buttons for single selection.
- SelectField – Dropdown list for single selection.
- TextAreaField – Multi-line text input field.
- FileField – File upload field.

To use Flask-WTF, we first need to install it using pip and import it into our application.
- pip install flask-WTF

Flask - How to Upload File in Python:

File uploading is a typical task in web apps. Taking car of file upload in Flask is simple all we need is to have an HTML form with the encryption set to multipart/form information to publish the file into the URL.

The server-side flask script brings the file from the request object utilizing the request.files[] Object. On effectively uploading the file, it is saved to the ideal location on the server.

Flask - CSRF Protection:

It is a security mechanism that prevents Cross-Site Request Forgery attacks by ensuring that requests originate from the authenticated user and not from a malicious third-party site.

This can lead to unauthorized actions being performed on behalf of the user, such as changing account settings or making transactions.

How to Prevent CSRF Attacks?

One of the most effective ways to prevent CSRF attacks is by using CSRF tokens. These tokens are unique, dynamically generated values included in forms and verified by the server when a request is made.

Since attackers cannot predict these tokens, they are unable to forge valid requests.

Flask – Configuration:

Flask provides configuration support to control application behaviour using settings like debug mode, secret keys and environment variables, and allows organizing configuration for development and production environments.

Flask application frequently needs configuration for various settings to ensure that it functions correctly. These settings manage aspects such as database connections, security, sessions handling and more.

Flask provide a simple way to manage configurations using the app.config dictionary.

Syntax:

1. The simplest way to configure a Flask app is by directly assigning values to app.config –
   app.config['CONFIG_NAME']='value'
   Parameters:
- CONFIG_NAME – The name of the setting we want to define. Flask has built-in keys like DEBUG, SECRET_KEY and SQLALCHEMY_DATABASE_URL, but we can also create custom ones.
- value – The actual setting, which can be a string, Boolean, integer or even a dictionary.
2. Alternatively, Flask allows configurations to be loaded from an external file, such as config.py –
   app.config.from_pyfile('config.py')

Common Flask Configurations:

Configuration in Flask refers to setting up parameters that control various aspects of the application. These includes –

- Security settings – such as secret keys and session handling
- Database settings – to connect and manage databases
- Debugging options – to enable automatic reloading and error reporting
- Session management – for handling user sessions

- File handling and uploading – configure file storage

Let's look at some most common app configurations in Flask one by one.

Setting Up a Secret Key:

A Secret key is crucial for security-related functions in Flask, such as protecting session cookies and securing form submissions.

app.config['SECRET_KEY'] = 'your_secret_key'

This key should always be kept private and unique. In a production environment, it's recommended to store it in an environment variable instead of hardcoding it in the script.

Configuring a Database:

Most Flask applications require a database. Flask supports SQLALchemy for database management and we configure it using –

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite://database.db'

app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

- The SQLALCHEMY_DATABASE_URI defines the database type and location. Here, we're using an SQLite database stored in a file named database.db
- The SQLALCHEMY_TRACK_MODIFICATIONS settings is set to Flase to improve performance by disabling tracking of modifications to objects.

If you're using a different database, such as PostgreSQL or MySQL, the URI changes accordingly –

app.config['SQLALCHEMY_DATABASE_URI'] =
'postgresql://username:password@localhost/db_name'

Session Management Configuration:

Flask provides session management to store user-related information across multiple requests. The default session type stores data in cookies, but we can configure it for better security and control.

app.config['JSON_SORT_KEYS'] = False

app.config['JSONIFY_PRETTYPRINT_REGULAR'] = True

- JSON_SORT_KEYS = False – Prevents automatic sorting of keys in JSON response, preserving the order in which data is added.
- JSONIFY_PRETTYPRINT_REGULAR = True – ensures the JSON output is formatted in a human-readable way.

Loading Configuration from a File:

Instead of setting configuration inside app.py, we can also store them in a separate configuration file named config.py

```
# config.py
SECRET_KEY = ' your_secret_key'
SQLALCHEMY_DATABASE_URI = 'sqlite://database.db'
DEBUG = True
```

Then we load it into Flask app using:

- app.config.from_pyfile('config.py')

Enabling Debug Mode:

During development, enabling debug mode helps catch errors quickly by allowing automatic reloading of the server and displaying detailed error messages.

- app.config['DEBUG'] = True

File Upload Configuration:

If our application allows users to upload files, we must specify a folder to store these files –

- app.config['UPLOAD_FOLDER'] = 'uploads/'
- app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024 # 16MB Limit

The UPLOAD_FOLDER – specifies where uploaded files will be stored.

The MAX_CONTENT_LENGTH – limits the file upload size(here, 16MB). This prevents users from uploading excessively large files.

Flask Environment Specific Configurations:

When developing a Flask application, different enivironment such as development, testing and production require different settings.

Flask allows us to configure our application based on the enivironment, ensuring smooth transitions from development to deployment.

Steps to Set Environment-Specific Configurations:

Flask provides multiple ways to configure the application based on the environment. The most common methods include –

1. Using Configuration Objects (e.g., config.py)
2. Loading from Environment Variables
3. Using Flask.config.from_object() or Flask.config.from_envvar()

Let's go through each method in detail

Using Configuration Objects:

A common approach is to create a separate config.py file and define different configurations as Python classes.

```
# config.py
class  Config:
        DEBUG = False
        TESTING = False
        SECRET_KEY = 'your_secret_key'
        SQLALCHEMY_DATABASE_URI = 'sqlite:///default.db'

class DevelopmentConfig(Config):
        DEBUG = True
        SQLALCHEMY_DATABASE_URI = 'sqlite:///development.db'

class TestingConfig(Config):
        TESTING = True
        SQLALCHEMY_DATABASE_URI = = 'sqlite:///testing.db'

class ProductionConfig(Config):
        SQLALCHEMY_DATABASE_URI = = 'postgresql://user:password@localhost/production.db'
```

Explanation:

- Config is a base class that holds default settings
- DevelopmentConfig, TestingConfig and ProductionConfig inherit from Config and override specific settings based on the environment.

Now to apply a specific configuration in our Flask app we can use –

```
from flask import Flask
from config import DevelopmentConfig, ProductionConfig, TestingConfig
app = Flask(__name__)
app.config.from_object(DevelopmentConfig)  # Change based on environment
```

- Here we have applied the DevelopmentConfig environment to the app and similarly, using app.config.from_object() we can apply different environment settings whenver required.

Loading Configuration from Environment Variables:
We can also set environment-specific configurations using system environment variables. This is useful when deploying the application to cloud services where secrets and database credentials should not be hardcoded. Here's an example –

```
import os
from flask import Flask
app = Flask(__name__)
app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY', 'default_secret')
app.config['SQLALCHEMY_DATABASE_URI'] = os.environ.get('DATABASE_URL', 'sqlite:///default.db')
```

Now we can set the environment variables in the terminal using these commands –

- export SECRET_KEY='my_production_secret'
- export DATABASE_URL='postgresql://user:password@localhost/prod_db'

Using Flask.config.from_object() and Flask.config.from_envvar():
Flask provides built-in methods to load configurations dynamically. Instead of manually specifying configurations in the code, we can use from_object() and from_envvar().

from_object():
This method loads configuration settings from a Python object or module, making it useful when managing multiple environment configurations.

- app.config.from_object('config.ProductionConfig') # Loads settings from ProductionConfig

Here, config.ProductionConfig refers to the ProductionConfig class inside the config.py file. we can change this dynamically by passing a different class, such as DevelopmentConfig or TestingConfig, based on the environment.

from_envvar():
Instead of hardcoding configuration file names, we can store the file path in an environment variable and use from_envvar() to load it.

```
import os
app.config.from_envvar('FLASK_CONFIG_FILE')
```

Before running the app, set the environment variable in the terminal using command :

- export FLASK_CONFIG_FILE='/path/to/config.py'

Flask – Database:

Flask provides support for multiple databases for storing and managing application data. It can work with relational databases like SQLite, PostgreSQL and MySQL using Flask-SQLAlchemy, connect to MongoDB and execute raw SQL queries.

Flask- SQLAlchemy Database:

Flask doesn't have a built-in way to handle databases, so it relies on SQLAlchemy, a powerful library that makes working with databases easier. SQLAlchemy provides an Object Relational Mapper(ORM), allowing developers to interact with databases using Python code instead of raw SQL.

This brings several advantages –

- Simplifies database management
- Improve security
- Supports multiple database systems like SQLite, MySQL and PostgreSQL
- Easily integrates with Flask using the Flask-SQLAlchemy extension

Here, we need to install Flask and Flask-SQLAlchemy in it using command –

- pip install flask flask-sqlalchemy

In sqlalchemy, we use classes to create our database structure. In our application, we will create a Profile table that will be responsible for holding the user's id, first name, last name, and age.

Flask-SQLAlchemy Queries Cheat Sheet:

Operation	Example	Notes/SQL Equivalent
Get all records	profiles = Profile.query.all()	SELECT * FROM profile;
Get first record	profiles = Profile.query.first()	SE;LECT * FROM profile LIMIT 1;
Get by primary key	profiles = Profile.query.get(1)	Fetch by id
Filter by exact value	Profile.query.filter_by(first_name="John").all()	SELECT * FROM profile WHERE first_name="john";
Filter with conditions	Profile.query.filter(Profile.age>=18).all()	Support operators (>, <, >=, <=, !=)
Filter with multiple conditions	Profile.query.filter(Profile.age >= 18, Profile.first_name =="John").all()	Comma = AND
LIKE / Pattern Matching	Profile.query.filter(Profile.first_name.like("J%")).all()	Names starting with "J"
Get single record	profile = Profile.query.filter_by(first_name="John").first()	Returns first match
Get one or more	profile = Profile.query.filter_by(first_name="John").one_or_none()	Returns object or None, error if multiple
Count records	count = Profile.query.count()	SELECT COUNT(*) FROM profile;
Order results ascending	Profile.query.order_by(Profile.age).all()	Order by column
Order results descending	Profile.query.order_by(Profile.age.desc()).all()	
Limit results	Profile.query.limit(5).all()	First 5 rows
Offset / Skip rows	Profile.query.offset(5).limit(5).all()	Skip first 5, get next 5

Combine filter + order + limit	Profile.query.filter(Profile.age>=18).order_by(Profile.first_name).limit(10).all()	All combined
Update a record	profile = Profile.query.get(2) profile.age = 25 db.session.commit()	Modifies existing record
Delete a record	profile = Profile.query.get(3) db.session.delete(profile) db.session.commit()	Deletes record from DB
Filter with OR	from sqlalchemy import or_ Profile.query.filter(or_(Profile.age<18, Profile.first_name=="John")).all()	Either condition matches
Filter with AND	from sqlalchemy import and_ Profile.query.filter(and_(Profile.age>=18, Profile.first_name=="John")).all()	Both conditions must match
Distinct	Profile.query.distinct(Profile.first_name).all()	Unique values of column
Save changes	Db.session.commit()	

Always commit after add(), update() or delete()

- db.session.commit()

Use app context when creating tables or running scripts outside Flask routes.

- with app.app_context():
  db.create_all()

<u>The table below explains some of the keywords used in the model class:</u>

Column	used to create a new column in the database table
Integer	An integer data field
primary_key	If set to True for a field ensures that the field can be used to uniquely identify objects of the data table.
String	An string data field. String(<maximum length>)
unique	If set to True it ensures that every data in that field in unique.
nullable	If set to False it ensures that the data in the field cannot be null.
__repr__	Function used to represent objects of the data table.

<u>How to Build a Web App using Flask and SQLite in Python:</u>

We'll create a Flask app that takes user input through a form and displays it on another page using SQLite.

Run the following commands to install Flask and SQLite –

- pip install flask
- pip install db-sqlite3

Connect to the database using -

- sqlite3.coonect('database.db')

Selects all columns from the table and retrives data using –

- fetchall()

Making a Flask app using a PostgreSQL database:

The Postgres databses can be accessed via one of two methods in Python. Installing PgAdmin4 is the first step because it offers a user interface for interacting with databases and another for using the psycopg2 connector.

In this post, we'll concentrate on a different approach that lets us alter the Postgres databases: using the psycopg2 connector.

Psycopg2 Cheat Sheet:

Task	Code Example	Notes
Connect DB	psycopg2.connect(...)	Create connection object
Create Table	CREATE TABLE IF NOT EXISTS ...	Run via execute()
Create Cursor	cur = conn.cursor()	Used to run queries
Execute Query	cur.execute("SELECT * FROM products")	Basic execution
Parameterized Query	cur.execute("SELECT * FROM products WHERE id=%s", (1,))	Prevent SQL injection
Fetch One	cur.fetchone()	Returns single row
Fetch All	cur.fetchall()	Returns all rows
Fetch Many	cur.fetchmany(5)	Limited rows
Insert	cur.execute("INSERT INTO products (name, price) VALUES (%s, %s)", ("Apple", 1.99))	Then commit
Update	cur.execute("UPDATE products SET price=%s WHERE id=%s", (2.5, 1))	Then commit
Delete	cur.execute("DELETE FROM products WHERE id=%s", (1,))	Then commit
Commit	conn.commit()	Save changes
Rollback	conn.rollback()	Undo on error
Close Cursor	cur.close()	Cleanup
Close Connection	conn.close()	Cleanup
Context Manager	with psycopg2.connect(...) as conn:	Auto close
Get Inserted ID	RETURNING id + fetchone()	Useful for APIs
Row Count	cur.rowcount	Rows affected
Debug Query	print(cur.query)	Shows last SQL

We can use string %S to %s.

Sending Data from a Flask app to MongoDB Database:

MongoDB is an open-source database that stores data in JSON -like documents. It is classified as a NoSQL database because it is based on different fundamentals as compared to a SQL relational database.

MongoDB Cheat sheet(Flask):

Task	Code
Install	pip install flask pymongo flask-pymongo
Import	from flask_pymongo import PyMongo
Init App	app = Flask(__name__)

Config DB	app.config[“MONGO_URL”] = “mongodb://localhost:27017/mydb
Init Mongo	mongo = PyMongo(app)
Get DB	db = mongo.db
Get Collection	users = db.users
Insert one	users,insert_one({“name”:”John”})
Insert Many	users,insert_many({“name”:”A”},{“name”:”B”})
Find one	users.find_one({“name”:”John”})
Find All	users.find()
Condition	user.find({“age”:{“$gt”:20}})
Update One	users.update_one({“name”:”John”},{“$gt”:{“age”:30}})
Update Many	users.update_many({"age":{"$lt":18}},{"$set":{"minor":True}})
Delete One	users.delete_one({"name":"John"})
Delete Many	users.delete_many({"age":{"$lt":18}})
$gt - Greater than $lt - Less than $gte – Greater than equal $lte – Lessthan equal $ne – Not equal $in – In list	{"age": {"$gt": 20}} {"age": {"$lt": 20}} {"age": {"$gte": 18}} {"age": {"$lte": 30}} {"age": {"$ne": 25}} {"age": {"$in": [20,25]}}
Sort Desc	users.find().sort("age", -1)
Sort Asc	users.find().sort("age", 1)
Limit	users.find().limit(5)
Imort ObjectId Handling	from bson.objectid import ObjectId
Find by Id	users.find_one({"_id": ObjectId(id)})

Login and Registration Project in Flask using SQL:

Creating user authentication system is a fundamental part of web applications.

Flask + MySQL Cheat Sheet:

Task	Code
Install	pip install flask flask-mysqldb
Import	from flask_mysqldb import MySQL
Init	mysql = MySQL(app)
MYSQL_HOST	localhost
MYSQL_USER	root
MYSQL_PASSWORD	password
MYSQL_DB	mydb
Create DB	CREATE DATABASE mydb;
Use DB	USE mydb;
Create Table	CREATE TABLE users(id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100), email VARCHAR(100));
Drop Table	DROP TABLE users;
Insert	cursor.execute("INSERT INTO users(name,email) VALUES(%s,%s)", (name,email))
Select All	cursor.execute("SELECT * FROM users")
Select One	cursor.execute("SELECT * FROM users WHERE id=%s",(id,))
Update	cursor.execute("UPDATE users SET name=%s WHERE id=%s",(name,id))

Delete	cursor.execute("DELETE FROM users WHERE id=%s",(id,))
fetchone()	Get single row
fetchall()	Get all rows
Commit	mysql.connection.commit()
Close cursor	cursor.close()

How to Execute Raw SQL in Flask - SQLAlchemy App:
In a Flask application that uses SQLAlchemy, we usually interact with the database using Python objects and methods. However, there are times when we need to execute raw SQL queries directly.

Installing requirements:
- pip install Flask flask_sqlalchemy pymysql cryptography

Sytnax:
To run raw SQL queries, we first create a flask-SQLAlchemy engine object using which we can connect to the database and execute the SQL queries. The syntax is –
- flask_sqlalchemy.SQLAlchemy.engine.execute(statement)

Flask – Middlewares:
Middlewares in Flask process requests and responses before they reach the application or the client. In Flask, middleware functions sit between the client request and the final response, enabling tasks such as logging, authentication and modifying requests or responses.

Let's explore several ways to implement middleware in Flask applications –
Creating Custom Middleware in Flask:
Flask provides hooks, which are special functions that allows you to execute code before or after processing a request. These hooks help in modifying requests, logging, authentication, and more. Two commonly used hooks for middleware are –
- before_request – Runs before the request is processed.
- after_request – Runs after the request is processed, modifying the response if needed.

Middleware for Authentication and Authorization:
Authentication middleware ensures that only authorized users can access specific routes. This is useful for securing APIs and web applications.

Third-Party Middleware in Flask:
Instead of writing custom middleware for every feature, we can use Flask extensions or third-party libraries for advanced middleware functionalities. Some commonly used middleware libraries include –
- Flask-CORS: Handles Cross-Origin Resource Sharing(CORS).
- Flask-Limiter: Implements rate limiting to prevent abuse.
- Flask-Talisman – Adds security headers for better protection.

What is CORS:
It is a security feature implemented by web browsers that restricts how resources on a web page can be requested from another domain.To use any third party middleware in our application, we are required to install its module in our enivironment, we use this command in terminal to install –
- pip install flask-cors

Flask – WSGI Middleware:

When building Flask applicants, middleware helps modify requests and responses before they reach the application or the client.

Flask itself supports middleware using hooks like before_request and after_request, but WSGI(Web Server Gateway Interface) middleware works at a lower level. It sits between the web server and the application, intercepting and modifying requests and responses before they reach Flask.

This allows us to add features without modifying the Flask app itself, some of the features are –

- Logging
- Authentication
- Compression

How to Implement WSGI Middleware:

WSGI middlware follows a specific pattern –

- It is implemented as a class that takes the Flask app as an argument.
- The class must be define a __call__ method, which process the WSGI environment and start_response.
- The modified request is then passed to the Flask app.

Let's look at the Syntax to create and apply WSGI middleware to a Flask app.

```
Class CustomWSGIMiddle:
        def __init__(self, app):
                self.app = app          # Wraps the Flask app
        def __call__(self, environ, start_response):
                # Modify request before passing it to Flask
                print(f"Incoming request: {environ['Request_Method']}
{environ['Path_Info']}")

        # Process the request with the Flask app
        response = self.app(environ, start_response)

        # Modify response if needed
        return response
```

Now, to apply the above WSGI Middleware to a flask app use this –

app.wsgi_app = CustomWSGIMiddleware(app.wsgi_app)

Using Third-Party WSGI Middleware:

Instead of writing middleware, you can use pre-built WSGI middleware. A popular option is Werkzeug's ProxyFix, which helps handle reverse proxy headers (like Ngin or a load balancer). Reverse proxies often modify headers like REMOTE_ADDR, HTTP_HOST, and SCRIPT_NAME, which Flask wouldn't recognize correctly by default.

We use –

- app.wsgi_app = ProxyFix(app.wsgi_app, x_for=1, x_proto=1, x_host=1, x_port=1, x_prefix=1)

Here,

x_for=1 tells Flask to trust the first X-Forwarded-For header, which contains the client's actual IP.

x_host=1 ensures Flask uses the X-Forwarded-Host header to determine the correct host.

Flask – Authentication:

Authentication in Flask is used to verify user identity and manage secure access to applications. It includes login system, password hashing, session management and permission control.

How to Add Authentication to App with Flask-Login:

We can implement authentication, login/logout functionality in flask app using Flask-Login. To get started, install Flask, Flask-Login, Flask-SQLAlchemy, and Werkzeug using this command –

- pip install flask flask_sqlalchemy flask_login werkzeug

Flask-Login: Manages user sessions and authentication.
Flask-SQLAlchemy: Stores user data like username and passwords.
Werkzeug: Used for secure password hashing and verification.

Flask - Password Hashing with Bcrypt:

Password hashing is the process of converting plaintext into a hashed or encrypted format that cannot be easily reverse-engineered to reveal the original password.
Bcrypt is a popular hashing algorithm used to hash passwords. It is a password-hashing function that is absed on the Blow-fish cipher and is designed to be slow and computationally expensive, making it more difficult for attackers to guess or crack passwords.

Key Terminologies:

Password Hashing:

The process of converting a plaintext password into a hashed or encrypted format.

Bcrypt:

A password hashing function based on the Blowfish cipher.

Salt:

Random data that is used as additional input to a one-way function that hashes a password or passphrase.

Hashing Algorithm:

A mathematical function that converts a plaintext password into a fixed-length hash value.

Iterations:

The number of times a password is hashed using the bcrypt algorithm.

Stepwise Implement with Bcrypt in Flask:

Step 1: Install Flask-Bcrypt:

- pip install flask-bcrypt

Step 2: Import Flask-Bcrypt:

- from flask-bcrypt import Bcrypt

Step 3: Create a Bcrypt Object:

We need to create a Bcrypt object and pass our Flask app as an argument.

- bcrypt = Bcrypt(app)

Step 4: Hash a password:

We need to decode the hashed password using Python decode('utf-8') as the generate_password_hash() function returns a bytes object.We can hash a password using the generate_password_hash() function of the Bcrypt object.

- hashed_password = bcrypt.generate_password_hash('password').decode('utf-8')

Step 5: Verify a Password:

We can verify a password by using the check_password_hash() function. If function returns True if the password matches the hashed password, otherwise, it returns False.

- is_valid = bcrypt.check_password_hash(hashed_password, 'password')

Flask- Role Based Access Control(RBAC):

Role-Based Access Control(RBAC) is a security mechanism that restricts user access based on their roles within an application. Instead of assigning permissions to individual users, RBAC groups users into roles and each role has specific permissions.

Installing Packages:

After creating a Flask app, we need to install modules required in this project, to install them execute this command in the terminal –

- pip install flask flask-security flask-sqlalchemy flask-login email-validator

It will install these packages –

Flask – A lightweight web framework for building web applications

Flask-Security – Adds authentication and role management features.

Flask-SQLAlchemy – Integrates SQLAlchemy for easy database operations.

Flask-Login – Manages user sessions and handles login/logout.

Email-validators – Validates email addresses for correct formatting.

Flask- Session:

Sessions in Flask store user-specific data across requests, like login status, using cookies. Data is stored on the client side but signed with a secret key to ensure security. They help maintain user sessions without requiring constant authentication.

Installation:

After creating flask app, we need to install modules required in this project, to install them execute this command in the terminal –

- pip install flask flask-session

Flask – Using JWT for user authentication:

JWT(JSON Web Token) is a compact, secure, and self-contained token used for securely transmitting information between parties. It is often used for authentication and authorization in web applications.

A JWT consists of three parts –

1. Header – Contains metadata(e.g, algorithm used for signing)
2. Payload – Stores user information(claims like user ID, roles)
3. Signature – Ensures token integrity using a secret key.

In Flask, JWT is commonly used to authenticate users by issuing tokens upon login and verifying them for protected routes.

Installation and setting up flask:

- pip install PyJWT

How JWT Authentication Works in Flask:

User Logs In:

- User sends email+password
- Server verifies credentials
- If valid → server creates a JWT

```
token = jwt.encode({'public_id': user.public_id,'exp': datetime.utcnow() + timedelta(hours=1},
app.config['SECRET_KEY'], algorithm='HS256')
```

Here, The token includes:

- public_id → identifies user
- exp → expiry time

Token is Sent to Client:

You typically store it in :

- cookies
- OR headers

response.set_cookie('jwt_token', token)

Client Sends Token in Requests:

Each protected request includes the token:

- Automatically via cookies
- Or manually in headers

Server Verifies Token:

This is done using a decorator like your token_required:

data = jwt.decode(token, app.config['SECRET_KEY'], algorithms=['HS256'])

Flask – Cookies:

Cookies store user data in the browser as key-value pairs, allowing websites to remember logins, preferences, and other details. This helps improve the user experience by making the site more convenient and personalized.

Setting Cookies in Flask:

set_cookie() method:

Using this method we can generate cookies in any application code. The syntax for this cookies setting method –

- Response.set_cookie(key, value='', max_age=None, expires=None, path='/', domain=None, secure=None, httponly=False)

Parameters:

- key – Name of the cookie to be set.
- value – Value of the cookie to be set.
- max_age – should be a few seconds, None(default) if the cookie should last as long as the client's browser session.
- expires – should be a datetime object or UNIX timestamp.
- domain – To set a cross-domain cookie.
- path – Limits the cookie to given path, (default) it will span the whole domain.

Example:

```
from flask import Flask, request, make_response
app = Flask(__name__)

#Using set_cookie() mthod to set the key-value pairs below.
@app.route('/setcookie')
def setcookie():
        # Intializing response object
        resp = make_response('Setting the cookie')
        resp.set_cookie('GFG', 'ComputerScience Portal')
        return resp
app.run()
```

- here, make_response creates a response object to attach cookies.

Getting Cookies in Flask:
cookies.get() method:
This method retrieves the cookie value stored from the user's web browser through the request object.

```
from flask import Flask, request, make_response
app = Flask(__name__)

# getting cookie from the previous set_cookie code
@app.route('/getcookie')
def getcookie():
	GFG=request.cookies.get('GFG')
	Return 'GFG is a'+ GFG
app.run()
```

How to return a JSON response from a Flask API?
When building a REST API with Flask, JSON is the standard format used to send data between the server and client. It's simple, readable, and works well across platforms. Flask makes it easy to return JSON responses, which is useful for sending data like user info, product details or status messages to web or mobile apps.
For example, if the API wants ro return user data, the JSON response might look like this –

```
{	"name":"Lilly",
	"age":25
}
```

Before building a Flask REST API, make sure Flask library is already installed. Also to install the Flask-restful library, use below command –

- pip install Flask-RESTful

There are two ways to build a simple Flask REST API that returns a JSON response.
Using Flask jsonify object:
jsonify() Function:
It converts Python data(like dictionaries or lists) into a JSON response. It automatically sets the Content-Type to application/json, so the client knows the response is in JSON format.
Follow these steps to create the API using jsonify:

- Create a new python file named 'main.py'
- Import Flask, jsonify and request from the flask framework.
  from flask import Flask, request, jsonify
- Initialize the Flask app –
  app = Flask(__name__)
- Define a function named ReturnJSON() to return a sample JSON response.
- Create a route for the function using –
  @app.route('/path_of_the_response', methods=['GET'])
- Inside the function, check if request method is GET. Then, create a Python dictionary and return it as a JSON response.
  if request.method == 'GET':
  	data = {"message": "Hello, this is a JSON response!"}
  	return jsonify(data)
- Run the Flask app.

Using flask_restful library with Flask:

flask_restful:

flask_restful is a Flask extension. It uses classes(Resource) instead of route decorators, making the code cleaner and more organized. Each class handles HTTP methods like GET, POST, etc.. making this approach ideal for larger or more structured APIs.

Follow these steps to create the API using flask_restful:

- Create a new python file named 'main.py'
- Import Flask from the flask framework
- Import API and Resource from the 'flask_restful' library
- Initialize the Flask app –
  app = Flask(__name__)
- Create an API object using –
  api = Api(app)
- Define a resource class called ReturnJSON [class returnjson(Resource)], and create a get() method inside it that returns a JSON response as a dictionary.
- Add this resource to the API using –
  api.add_resource(ReturnJSON, '/returnjson')
- Run the Flask app

Flask – Rest APIs:

Flask makes building REST APIs easy by handling requests like GET, POST, PUT and DELETE. With Flask-RESTful or simple routes, one can send and receive JSON data, manage authentication and secure the API.

Flask Creating Rest APIs:

A REST API(Representational State Transfer API) is a way for applications to communicate over the web using standard HTTP methods. It allows clients to interact with a server by sending requests and receiving responses, typically in JSON format.

REST APIs follow a stateless architecture, meaning each request from a client to the server is independent and does not rely on previous requests. This makes REST APIs scalable, flexible, and easy to integrate with different platforms.

Key Features of REST APIs:

- Uses standard HTTP methods for CRUD operations:
  GET – Retrives data
  POST – Send new data
  PUT – Update existing data
  DELETE – Remove data
- Follows a stateless design(each request is processed independently).
- Uses JSON or XML for structured data exchange.
- Enables easy integration with web, mobile, and third-party services.

Flask – RESTFUL Extension:

Flask-RESTful is an extension for Flask that simplifies the process of building REST APIs. It provides additional tools for creating APIs with minimal boilerplate code while following REST principles. With Flask-RESTful, defining API resources, handling requests, and formatting responses become more structured and efficient.

Features of Flask-RESTful:

- Simplifies API Development – Provides a cleaner and more structured approach.

- Automatic Request Parsing – Built-in support for handling request arguments.
- Better Resource Management – Uses classes to define API resources.
- Response Formatting – Automatically formats responses in JSON.
- Integrated Error Handling – Provides built-in exception handling for common errors.

Key concepts in Flask-RESTful:

Resource Class – Each API resource is defined as a class that inherits from Resource.

Methods for HTTP Requests – get(), post(), put() and delete() methods handle different HTTP requests.

Adding Resource to API – The add_resource() method like resource classes to specific URL endpoints.

Basic Example:

```
from flask import Flask, request
from flask_restful import Api, Resource
app = Flask(__name__)
api = Api(app)
# Define a simple resource
class GFG(Resource):
	def get(self):
		return {“message”:”Hello, Welcome to Flask-RESTful”}
# Adding resource to API
api.add_resource(GFg,’/’)
if __name__ == “__main__”:
	app.run(debug=True)
```

Action	**Method**	**URL**	**Body(JSON)**	**Response**
Get all books	GET	/books	None	List of books
Get a book by ID	GET	/books/1	None	Book details or 404
Add a new book	POST	/books	{“id”:4, “title”:”New”, “author”:”X”}	New book data
Update a book	PUT	/books/2	{“title”:”Updated	Updated book data or 404

Flask – Serialization and Deserialization:

Serialization and deserialization are fundamental concepts in web development especially when working with REST APIs in Flask.

Serialization refers to converting complex data types (such as Python objects) into a format that can be easily stored or transmitted, like JSON or XML.

Deserialization, on the other hand, is the process of converting serialized data back into Python objects.

Flask provides several ways to handle serialization and deserialization efficiently, wheatheer using built-in modules like json, third-party libraries like marshmallow or Flask-RESTful’s request parsing.

Significance of Serialization and Deserialization:

Data Exchange – APIs need to send and receive data in a structured format.

Security – Proper serialization prevents data tampering and ensures data integrity.

Validation – Deserialization allows for data validation before processing it.

Performance – Optimized serialization improves API performance.

Serialization in Flask:

Using Flask's jsonify:

jsonify() Function – which automatically serializes Python dictionaries and lists into JSON.

```
from flask import Flask, jsonify
app = Flask(__name__)
@app.route('/data):
def get_data():
        data = {"name":"Alice", "age":35, "city":"New York"}
        return jsonify(data)
if __name__ == "__main__":
        app.run(debug=True)
```

Using Python's json Module:

We can also use Python's built-in json module for manual serialization, such as logging or saving data. It's useful in scenarios where Flask's jsonify isn't required.

```
import json
data = { "name":"David", "age":25}
json_data = json.dumps(data)    # Converts Python dict int JSON string
print(json_data)
```

Deserialization in Flask:

It is reverse process of serialization. It allows a Flask application to read and process incoming JSON data, converting it into a usable Python object.

Using request.get_json():

Flask's request.get_json() automatically parses incoming JSON requests, making it easy to extract and process data.

```
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.route('/submit', methods=['POST'])
def receive_data():
        data = request.get_json()
        return jsonify({"message":"Data received", "data": data})
if __name__ == "__main__":
        app.run(debug=True)
```

Using marshmallow for serialization and Deserialization:

While Flask provides basic serialization and deserialization capabilities, marshmallow enhances them with data validation and structured schemas, ensuring data integrity.

Install marshmallow using this command –

- pip install flask-marshamallow

To demonstrate the working of marshmallow let's create a Flask app in which we define a UserSchema using marshmallow, which ensures that the incoming JSON data includes a valid username and email. The schema validates the request before processing it, reducing errors and improving data integrity.

```
from flask import Flask, request, jsonify
from marshmallow import Schema, fileds
App = Flask(__name__)
# Define a Schmea
```

```
class UserSchema(Schema):
        username = fields.Strings(required=True)
        email = fields.String(required=True)
user_schema = UserSchema()
@app.route('/validate', methods=['POST'])
def validate_user():
        json_data = request.get_json()
        errors = user_schema.validate(json_data)

        if errors:
                return jsonify(errors), 400        # Return validation errors
                return jsonify({"message":"Valid data", "data": json_data})
if __name__ == "__main__":
        app.run(debug=True)
```

Explanation:

- /validate route accepts POST requests
- request.get_json() extracts incoming JSON data.
- user_schema.validate(json_data) checks if the provided data matches the schema.
- Return the error messages with an HTTP 400 (Bad Request) status if the validation fails.
- If the data is valid, return a success message along with the provided data.

Advanced Topics:

Flask also supports advanced features for building scalable applications, including asynchronous programming, modular application structure and real-time communication.

Flask Asynchronous Programming Using async.io:

Flask is inherently synchronous, meaning each request is processed one at a time. However, modern web applications often require handling multiple tasks simultaneously, such as –

- Making external API calls
- Processing large datasets
- Managing real-time communication

Asynchronous programming allows us to execute multiple tasks concurrently, improving performance and responsiveness.

Features of asynchronous programming:

Improves Performance – Handles multiple tasks without waiting for each to complete.

Non-Blocking I/O – Ideal for tasks like fetching external data or database operations.

Better User Experience – Reduces delays for user when making API requests.

Syntax of async.io Programs:

Declaring an Asynchronous Function:

In Python, asynchronous functions are defined using the **async def** keyword.

```
import asyncio
async def my_async_function():
        print("Task started")
        await asyncio.sleep(2)   #Simulates an async operation
        print("Task completed")
# Running the async function
asyncio.run(my_async_function())
```

Explanation:
async def – Declares a function as synchronous, meaning it can perform non-blocking operations.
await – Suspends execution of the function until the awaited task completes, preventing blocking.

Running Multiple Async Tasks:
asyncio.gather() – To run multiple async functions concurrently.

```
async def task_1():
    await asyncio.sleep(2)
    return "Task 1 Completed"
async def task_2():
    await asyncio.sleep(3)
    return "Task 2 Completed"
async def main():
    results = await asyncio.gather(task_1(), task_2())        # Runs both tasks concurrently
    print(results)
asyncio.run(main())
```

Explanation:
async def function must be awaited inside another async function.
await asyncio.sleep(n) simulates non-blocking behaviour.
asyncio.gather() executes multiple async function in parallel.

Now that we have the fundamental understanding of how asynchronous programming works, let's understand how we can implement it in Flask applications with some basic flask app examples.
Asynchronous Database Queries in Flask:
Flask's traditional database extensions like Flask-SQLAlchemy are synchronous. To perform asynchronous database operations, we can use Tortise-ORM, an async ORM for Python.
Installation:

- pip install "flask[async]"
- pip install tortoise-orm aiosqlite

Flask-Web Sockets:
WebSockets enable real-time, bidirectional communication between a client and server. Unlike traditional HTTP requests, which follow a request-response cycle, WebSockets maintain a persistent connection, allowing instant data exchange.
This is particularly useful for applications like –

- Live chat
- Notifications
- Stock price updates
- Collaborative editing tools.

Flask, being a lightweight framework, does not provide WebSocket support by default, but with the help of Flask-SocketIO module, we can easily integrate WebSockets into a Flask application.
Installation:
Initialize Flask app and install required packages using the following command –

- pip install flask flask-socketio

Flask-SocketIO builds on top of Flask and provides a simple interface for managing WebSocket connections. A typical Flask WebSocket application involves –

- Initailizing the Flask app.
- Wrapping the Flask app with SocketIO.

- Defining event handlers using decorators such as @socketio.on().
- Running the app using socketio.run(app)

Key Components:

SocketIO(app) – This wraps the Flask application to enable WebSocket capabilities.
Event Handlers – Functions decorated with @socketio.on('<event_name>') respond to specific WebSockets events.
send() – Sends a simple message.
emit() – Sends a message to a specific event channel, which can include more complex data.

Initialization of Flask-SocketIO to a Flask application:

```
from flask import Flask, render_template
from flask_socketio import SocketIO

app = Flask(__name__)
app.config['SECRET_KEY'] = 'secret!'
socketio = SocketIO(app)

if __name__ == "__main__":
	socketio.run(app)
```

We can also use init_app() style of initialization is also supported:

```
from flask import Flask, render_template
from flask_socketio import SocketIO

socketio = SocketIO()

def create_app():
	app = Flask(__name__)
	app.config['SECRET_KEY'] = 'secret!'
	socketio.init_app(app)
	return app

if __name__ == "__main__":
	app = create_app()
	socketio.run(app)
```

The application must be serve a page to the client that loads the Socket.IO library and establishes a connection –

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/socket.io/4.0.1/socket.io.js">
<script type="text/javascript" charset="utf-8">
   var socket = io();
   socket.on('connect', function() {
      socket.emit('my event', {data: 'I\'m connected!'});
   });
</script>
</script>
```

Receiving Messages:

When using SocketIO, messages are received by both parties as events. On the client side javascript callbacks are used. With Flask-SocketIO the server needs to register handlers for these events, similarly to how routes are handled by view functions.

The following example creates a server-side event handlers for an unnamed event use string data –

```
@socketio.on('message')
def handle_message(data):
    print('received message:' + data)
```

The following example creates a server-side event handlers for an unnamed event use JSON data -

```
@socketio.on('json')
def handle_json(json):
    print('received json: '+str(json))
```

The most flexible type of event uses custom event names. The message data for these events can be string, bytes, int, or JSON –

```
@socketio.on('my event')
def handle_my_custom_event(json):
    print('received json: ' + str(json))
```

Custom named events can also support multiple arguments -

```
@socketio.on('my_event')
def handle_my_custom_event(arg1, arg2, arg3):
    print('received args: ' + arg1 + arg2 + arg3)
```

When the name of the event is a valid Python identifier that does not collide with other defined symbols, the @socketio.event decorator provides a more compact syntax that takes the event name from the decorated function –

```
@socketio.event
def my_custom_event(arg1, arg2, arg3):
    print('received args:' +arg1 + arg2 + arg3)
```

Named events are the more flexible, as they eliminate the need to include additional metadata to describe the messaage type. The names message, json, connect and disconnect are reserved and cannot be used for named events.

Flask-SocketIO also supports SocketIO namespaces, which allow the client to multiplex several independent connections on the same physical socket –

```
@socketio.on('my event', namespace='/test')
def handle_my_custom_namespace_event(json):
    print('received json:', + str(json))
```

when a namespace is not specified a default global namespace with the name '/' is used.
For cases when a decorator syntax isn't convenient, the on_event method can be used –

```
def my_function_handler(data):
    Pass
socketio.on_event('my event', my_function_handler, namespace='/test')
```

Sending Messages:

SocketIO event handlers defined as shown in the previous section can send reply messages to the connected client using the send() and emit() functions.

The following examples bounce received events back to the client that sent them –

```
from flask_socketio import send, emit
@socketio.on(‘message’)
def handle_message(message):
        send(message)
@socketio.on('json')
def handle_json(json):
        send(json, json=True)
@socketio.on('my event')
def handle_my_custom_event(json):
        emit('my response', json)
```

When working with name spaces, send() and emit() use the namespace of the incoming message by default. A different name space can be specified with the optional argument -

```
@socketio.on('message')
def handle_message(message):
  send(message, namespace='/chat')
@socketio.on('my event')
def handle_my_custom_event(json):
  emit('my response', json, namespace='/chat')
```

To send an event with multiple arguments, send a tuple –

```
@socketio.on('my event')
def handle_my_custom_event(json):
  emit('my response', ('foo', 'bar', json), namespace='/chat')
```

SocketIO supports acknowledgment callbacks that confirm that a message was received by the client-

```
def ack():
  print('message was received!')
@socketio.on('my event')
def handle_my_custom_event(json):
  emit('my response', json, callback=ack)
```

Boardcasting:

Another very useful feature of SocketIO is the broadcasting of messages. Flask-SocketIO supports this feature with the broadcast=True optional argument to send() and emit()-

```
@socketio.on(‘my event’)
def handle_my_custom_event(data):
        emit(‘my response’, data, broadcast=True)
```

The socketio.send() and socketio.emit() methods can be used to broadcast to all connected clients-

```
def some_function():
  socketio.emit('some event', {'data': 42})
```

Rooms:

Flask-SocketIO supports this concept of rooms through the join_room() and leave_room() functions-

```
from flask_socketio import join_room, leave_room
@socketio.on('join')
def on_join(data):
    username = data['username']
    room = data['room']
    join_room(room)
    send(username + ' has entered the room.', to=room)
@socketio.on('leave')
def on_leave(data):
    username = data['username']
    room = data['room']
    leave_room(room)
    send(username + ' has left the room.', to=room)
```

Connection Events:

Flask-SocketIO also dispatches connection and disconnection events -

```
@socketio.on('connect')
def test_connect(auth):            # Here auth argument in the connection handler is optional.
    emit('my response', {'data': 'Connected'})
@socketio.on('disconnect')
def test_disconnect(reason):
    print('Client disconnected, reason:', reason)
```

The connection event handler can return False to reject the connection, or it can also raise *ConnectionRefusedError*. This is so that the client can be authenticated at this point. When using the exception, any arguments passed to the exception are returned to the client in the error packet.

```
from flask_socketio import ConnectionRefusedError
@socketio.on(‘connect’)
def connect():
        if not self.authenticate(request.args):
                raise ConnectionRefusedusedError(‘unauthorized!’)
```

Class-Based Namespaces:

As an alternative to the decorator-based event handlers. The event handlers that belong to a namespace can be created as methods of a class. The flask_socketio.Namespace is provided as a base class to create class-based namespaces -

```
from flask_socketio import Namespace, emit
class MyCustomNamespace(Namespace):
    def on_connect(self):
        pass
    def on_disconnect(self, reason):
        pass
    def on_my_event(self, data):
        emit('my_response', data)
socketio.on_namespace(MyCustomNamespace('/test'))
```

Error Handling:
Flask-SocketIO can also deal with exceptions –

```
@socketio.on_error()            # Handles the default namespace
def error_handler(e):
        pass
@socketio.on_error('/chat')     # Handles the '/chat' namespace
def error_handler_chate(e):
        pass
@socketio.on_error_default      # handles all namespaces without an explicit error handler
def default_error_handler(e):
        pass
```

Flask – Blueprint:
The main idea behind Blueprints is to organize a Flask application into separate modules, each handling a specific feature with its own routes, templates, and static files. This helps keep the application structured and reduces complexity.
Steps to Use Blueprints:

- Create a blueprint in a separate module.
- Define routes inside that blueprint
- Register the blueprint in the main application.

Flask – Deployment and Error Handling:
After development, Flask applications must be deployed and properly handle errors. This section covers deployment methods and techniques for managing application erros.
Flask – Subdomain:
A subdomain is a child domain that's part of a larger domain. For example, practice.geeksforgeeks.org and write.geeksforgeeks.org are subdomains of the geeksforgeeks.org domain, which in turn is a subdomain of the org top-level domain(TLD).
Setting the Host Entries:
Step 1: Open Notepad with administrator access and open the host file of your system in it make edits, path to the host file for –

- Linux - /etc/hosts
- Windows – C:\\windows\\System32\\Drivers\\etc\\hosts

Step 2: Add entries for your custom domain and subdomain in the end of the host file and save changes –

```
127.0.0.1       vibhu.gfg
127.0.0.1       practice.vibhu.gfg
```

Example:

```
from flask import Flask
app = Flask(__name__)
app.config['SERVER_NAME'] = 'vibhu.gfg:5000'
@app.route('/')
def home():
   return "Welcome to GeeksForGeeks !"
if __name__ == "__main__":
   app.run()
```